

Proving the hard-to-round search

A plan in three programs, from `htr.c` with MPFR to a search with a proved exponential

Names set in `monospace` are identifiers of `code/APaul/htr.c` or of the Rocq files in `code/APaul/rocq/`. Every number is marked **read** (off the code or a file), **measured** (a run), or **computed** (arithmetic on the other two).

1 The goal

Let x be a double and e the exponent of $\exp x$, so that $Y(x) = \exp(x) \cdot 2^{54-e}$ lies in $[2^{53}, 2^{54})$. The integers are the **round-bit grid**: the doubles and the midpoints between them. We say that x is **hard to round** at level m when

$$\text{dist}(Y(x), \mathbb{Z}) < 2^{-m},$$

that is, when the m bits after the round bit of $\exp x$ are all equal.

The theorem we want. For an interval $[x_0, x_1)$, the list of candidates the program produces contains every double of $[x_0, x_1)$ that is hard to round at level m . We ask for a **superset**: the program may report false candidates, it must not miss a true one. Removing the false ones is a separate and easier question (Section 6).

The final aim is the whole range where `exp` gives neither an infinity nor a NaN, about $x \in [-745, 709.78]$. Below $|x| < 2^{-54}$ the answer is known by hand (`exp x` rounds to 1 or just below). What is left is 64 binades on each side, $2 \cdot 64 \cdot 2^{52} = 2^{59} \approx 5.8 \cdot 10^{17}$ doubles (**computed**).

2 How the search works

The search cuts $[x_0, x_1)$ into **chunks** of $n = 2^{20}$ consecutive doubles (**read**, **search**). On a chunk starting at x , with u the ulp of x ,

$$Y(x + iu) \approx Y(x) + i \cdot Y(x)u, \quad 0 \leq i < n,$$

because the derivative of `exp` is `exp`. Only the fractional part matters, kept on 64 bits:

- **A** approximates the fractional part of $Y(x)$, times 2^{64} ;
- **B** approximates the fractional part of the slope $Y(x)u$, times 2^{64} ;
- **E** is a window: 2^{64-m} , the target, plus a bound on how far the straight line drifts from the curve over the chunk, $Y(x)u^2n^2/2$.

The inner loop adds **B** to **A** n times and reports every i where **A** is within **E** of an integer (the test $\mathbf{A} < 2 \cdot \mathbf{E}$ after shifting by **E**). This loop, a comparison and an addition, is the whole cost: 99.99% of the run on $[0.25, 0.25001)$ (**measured**, the Capla benchmark).

The one idea of the proof. The true value and the line differ by at most (drift + errors of **A**, **B**). If the window is at least target + drift + errors, no hard case falls outside it.

3 Program 1: `htr.c`, with MPFR trusted

This is the program as it is. MPFR is used in three places (**read**):

function	what it does	needed for the superset
<code>dd_exp</code>	<code>exp x</code> at 161 bits, split into three doubles $h + l + s$	yes: it seeds each chunk
<code>ref_exp</code>	<code>exp</code> at both ends, to check e is the same on the interval	yes, once per interval
<code>check</code>	decides a candidate, at $53 + m$ bits	no

We take as an axiom what MPFR documents: `mpfr_exp` returns the correctly rounded value at the requested precision.

The obligations.

#	obligation	state
1a	<code>dd_exp</code> : $ h + l + s - \exp x \leq 2^{-158} \exp x$ roughly, from one rounding at 161 bits and three conversions	to do, mild
1b	the conversions to 64 bits: <code>ldexp</code> , <code>get_uint64</code> , the reduction of <code>l</code> into $[0, 1)$, the wrap modulo 2^{64}	to do, fiddly
1c	the window covers target + drift + the errors of 1a and 1b	to do, the hard point
1d	the inner loop reports exactly the i in the window	done: <code>Scan.v</code>
1e	the chunks cover $[x_0, x_1)$	done at the nat layer: <code>ScanAll.v</code>

On 1c. In `htr.c`, `E` is target + drift and nothing else (**read**). Three errors are not in it: the curvature is taken at the start of the chunk and not its maximum over the chunk; the two truncations of `B` add up over n steps; and `A` itself is truncated. Our notes of 2026-08-01 say they fit inside the margin; this has not been re-checked. The proof decides it. If they do not fit, the fix is one term added to `E`.

What `htr.c` does not handle. `search` asserts things that hold on $[0.25, 0.5)$ but not on the whole range (**read**):

- `get_uint64` asserts its argument is in $(-1, 1)$. The slope $Y(x)u$ is in $[2^{e_0}, 2^{e_0+1})$ where $x \in [2^{e_0-1}, 2^{e_0})$, so it passes only for $|x| < 1/2$ (**computed**). Only the fractional part of the slope matters, so the fix is to drop the integer part first.
- $[x_0, x_1)$ must sit in one binade of x and of $\exp x$ (asserted). The full range must be cut accordingly: about $2 \cdot 64$ binades of x and a cut every $\ln 2$ for $\exp x$ (**computed**).
- Near the ends, $\exp x$ becomes subnormal (below $x \approx -708$) or overflows (above $x \approx 709.78$). The definition of “hard to round” changes for subnormal outputs.
- The last chunk overruns x_1 ; the line `n = (x1 - x) / ux` is never used. This is harmless for a superset of $[x_0, x_1)$.

4 Program 2: an abstract evaluator

Program 2 is `htr.c` with `dd_exp` replaced by a parameter `eval`. The search itself (1b to 1e) is proved once, for any `eval` that meets a contract.

The contract. For every x in the range,

$$\text{eval}(x) = \tilde{y}, \quad |\tilde{y} - \exp x| \leq 2^{-\kappa} \exp x,$$

where $\tilde{y}$ is a fixed-point number with enough bits below the round bit (`htr.c` keeps 64 in `A`, **read**). From $\tilde{y}$ the search builds `A` and `B`, and the proof of 1c becomes: the window covers target + drift + $C(\kappa, n)$, an error term that depends only on κ and n .

How large must κ be. An error of $2^{-\kappa}$ relative is about $2^{54-\kappa}$ on the grid, so it is below one unit of `A` (i.e. 2^{-64}) when $\kappa \geq 118$ (**computed**). With $\kappa \approx 120$, the evaluator errors are small next to the drift, which is about 2^{48} units on $[0.25, 0.25001)$ (**computed** from `htr.c`’s formula). MPFR at 161 bits gives $\kappa \approx 158$, much more than needed.

What this buys. Program 1 becomes an instance: `eval = dd_exp`, and the contract follows from the MPFR axiom (1a). Program 3 is another instance, with no axiom.

5 Program 3: a proved evaluator

Now `eval` is a program proved against the contract. The first question is how fast it must be, and the answer is: not very.

Speed. `eval` is called once per chunk. The inner loop runs about $1.5 \cdot 10^9$ steps a second (**measured**, one P-core, turbo off), so a chunk takes about 0.7 ms (**computed**). An evaluator of 10 μs would add 1.4% (**computed**). On the full range there are $2^{59}/2^{20} = 2^{39} \approx 5.5 \cdot 10^{11}$ chunks (**computed**), and the whole search is about $3.9 \cdot 10^8$ core-seconds, 12 core-years (**computed**), whatever the evaluator. So the choice of evaluator is a question of **proof effort**, not of speed.

Three possible evaluators.

evaluator	how	certificates	difficulty
Tang, fixed point	$x = k \ln 2 + j/256 + r$; a table of 256 values $\exp(j/256)$; one polynomial of degree 10 for $\exp r$, $ r \leq 2^{-9.5}$ (error 2^{-130} , computed from the Taylor bound); integers on three 64-bit words	1 polynomial, 256 table entries, 1 reduction	3
pieces + section 5	one certified polynomial per piece, like <code>Cheb.v</code> ; the chunk seeds by exact additions (<code>Shift.v</code> , <code>Pdir_exp</code>)	$6 \cdot 10^5$ at degree 7, $4 \cdot 10^3$ at degree 15, for 2^{-100} (computed , formula checked at one point)	2 each, but volume
short Taylor near 0	for small $ x $, $\exp x = 1 + x + \dots$ with few terms, no table	1 polynomial per binade group	1–2

Recommendation. Tang for the bulk of the range: one proof, independent of the range. A different evaluator may be used where it makes the proof simpler, for instance the short Taylor series near 0, where no reduction is needed. Since speed does not depend on the evaluator, mixing them is only worth it for the proof.

The Tang obligations.

part	difficulty
the polynomial on r : one <code>interval</code> call, like <code>cheb_valid</code>	1
the 256 table entries: one small interval check each	1
the reduction: $\ln 2$ split on several words, $ k \leq 1075$	3
the sum of the truncation errors, one per product	2–3

6 Removing the false candidates

The superset needs no **check**. To get the exact list, each candidate is evaluated once at about 2^{-100} and decided. With Program 3 this is the same evaluator. The number of candidates matters here: about 41 per chunk on $[0.25, 0.25001)$ (**measured**: 7 056 503 over 171 799 chunks). The count comes from the width of the window, $2E/2^{64} \approx 2^{-14.6}$ per double (**computed**). A smaller n shrinks the drift quadratically and so the candidates, at the price of more chunks.

7 What to do, in order

1. **Program 1, 1a to 1c**, on $[0.25, 0.25001)$ where 1d and 1e are done. This settles the hard point, the window, with MPFR as the only axiom.
2. **Program 2**: state the contract and redo 1c against it, for any **eval**.
3. **Generalise the search** to the whole range: the slope above 1, the cuts at binades, the subnormal and overflow ends.
4. **Program 3**: the Tang evaluator and its proof, plus the short Taylor series near 0 if it simplifies things.
5. **The link to the running program**. The proofs above are about a Rocq model of the search. Tying them to the code that runs is a separate step; with Capla (our port of `htr_plain.c`, `code/APaul/capla/`) the program has a formal semantics, but how a Capla program is linked to a Rocq proof has not been tried.